

Gold artifact – DevOps

Case study: Appointment Booking System

1. Cel

Referencyjny, minimalny pipeline CI dla projektu booking system, który:

- instaluje zależności,
- uruchamia testy,
- raportuje sukces lub błąd,
- jest prosty, poprawny i powtarzalny.

2. Zakres

Pipeline obejmuje:

- checkout repozytorium,
- konfigurację Pythona,
- instalację zależności,
- uruchomienie testów,
- zapis wyniku joba.

Nie obejmuje:

- deploymentu produkcyjnego,
- buildów kontenerów,
- secrets management,
- environment promotion,
- monitoringu produkcyjnego.

3. Założenia

Przyjęto:

- platformę: GitHub Actions
- język: Python 3.11
- testy uruchamiane przez pytest
- zależności instalowane z requirements.txt

4. Struktura repozytorium

```
booking_system/
├── app.py
├── models.py
├── repository.py
├── services.py
├── requirements.txt
├── tests/
│   └── test_booking.py
└── .github/
    └── workflows/
        └── ci.yml
```

5. Plik requirements.txt

```
pytest==8.2.2
```

6. Gold pipeline – ci.yml

```
name: CI
```

```
on:
```

```
  push:
```

```
    branches: [ main, master ]
```

```
  pull_request:
```

```
    branches: [ main, master ]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout repository
```

```
        uses: actions/checkout@v4
```

```
      - name: Set up Python
```

```
        uses: actions/setup-python@v5
```

```
        with:
```

```
          python-version: '3.11'
```

```
      - name: Install dependencies
```

```
        run: |
```

```
          python -m pip install --upgrade pip
```

```
          pip install -r requirements.txt
```

- name: Run tests

run: |

pytest -q

7. Oczekiwane zachowanie pipeline

Za poprawne działanie uznaje się sytuację, w której:

- pipeline uruchamia się po push i pull_request,
- środowisko buduje się bez błędu,
- testy uruchamiają się automatycznie,
- wynik joba jest czytelny (success / failure).

8. Referencyjne przypadki użycia

D1. Poprawny projekt

- kod poprawny,
- testy poprawne,
- pipeline kończy się sukcesem.

D2. Błąd w kodzie

- testy wykrywają błąd,
- pipeline kończy się failure.

D3. Błąd w zależnościach

- brak pakietu lub zła wersja,
- pipeline kończy się failure na etapie instalacji.

D4. Błąd w YAML

- niepoprawna składnia workflow,
- pipeline nie uruchamia się poprawnie.

9. Referencyjny log sukcesu

Run pytest -q

.....

8 passed in 0.12s

10. Referencyjny log błędu testów

Run pytest -q

....F...

===== FAILURES

=====

FAILED tests/test_booking.py::test_booking_limit_enforced

1 failed, 7 passed in 0.10s

11. Referencyjny log błędu zależności

ERROR: Could not find a version that satisfies the requirement pytest==999.0.0

ERROR: No matching distribution found for pytest==999.0.0

12. Referencyjny log błędu YAML

Invalid workflow file

You have an error in your yaml syntax on line 12

13. Wymagane właściwości gold DevOps

Wersja referencyjna powinna być:

- poprawna składniowo,
- uruchamialna bez dodatkowej konfiguracji,
- czytelna,
- minimalna,
- wystarczająca do wykrycia błędów w kodzie i testach.

14. Minimalne metryki oczekiwane

Dla gold pipeline przyjmuje się:

- poprawne uruchomienie na czystym repo,
- automatyczne wykonanie testów,
- czytelny błąd w razie awarii,
- brak zbędnych kroków i zależności.

15. Co stanowi wynik referencyjny

Za poprawny gold DevOps uznaje się pipeline, który:

- działa na gold code i gold tests,
- kończy się sukcesem dla poprawnego projektu,
- kończy się błędem dla projektu z defektem,

- umożliwia prostą diagnozę przyczyny awarii.

16. Do czego używać tego gold

Ten artifact może być użyty jako:

- gold pipeline dla DevOps Team,
- część wspólnego gold project,
- punkt odniesienia dla oceny pipeline generowanych przez studentów i LLM.

17. Uwaga

Gold DevOps ma być minimalnym, poprawnym pipeline CI, nie pełnym rozwiązaniem produkcyjnym. Celem jest referencja do oceny jakości konfiguracji CI/CD i diagnozy błędów.